Bridge

dge · Draw shapes with Vector vs Raster renderers independently · Build with Wa

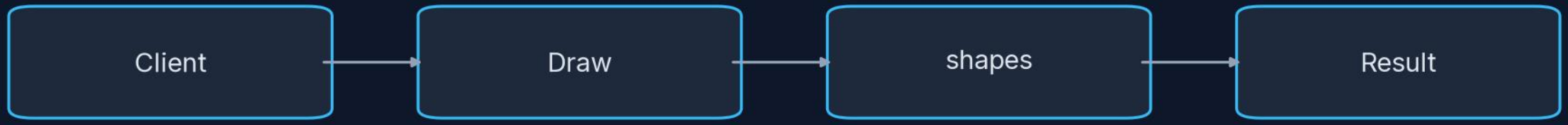
Today's Hook

- Problem: Draw shapes with Vector vs Raster renderers independently
- Pattern: Bridge — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Draw shapes with Vector vs Raster renderers independently
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Bridge: Draw shapes with Vector vs Raster renderers indepe

Real-World Scenario

- Running example: Draw shapes with Vector vs Raster renderers independently
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.structural.bridge;

/** Problem: support Vector and Raster rendering without exploding Shape×Renderer subclasses. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Bridge problem: shape x renderer ===");
    System.out.println("naive: VectorCircle, RasterCircle, VectorSquare, RasterSquare...");
    System.out.println(BridgeShapes.vectorCircle(3));
    System.out.println(BridgeShapes.rasterSquare(4));
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.gof.structural.bridge;

/** Public entry points for the bridge use-case demo. */
public final class BridgeShapes {
    private BridgeShapes() {}
    public static String vectorCircle(int r) { return new Circle(new VectorRenderer(), r).draw(); }
    public static String rasterSquare(int s) { return new Square(new RasterRenderer(), s).draw(); }
}
```

```java
package com.buildwithwaqar.patterns.gof.structural.bridge;

interface Renderer { String render(String shape); }

record VectorRenderer() implements Renderer {
    public String render(String shape) { return "vector:" + shape; }
}
record RasterRenderer() implements Renderer {
    public String render(String shape) { return "raster:" + shape; }
}

/** Abstraction decoupled from implementation via composition. */
public abstract class Shape {
    protected final Renderer renderer;
    protected Shape(Renderer renderer) { this.renderer = renderer; }
    public abstract String draw();
}
```

Solution (2)

```
final class Circle extends Shape {
    private final int radius;
    Circle(Renderer renderer, int radius) { super(renderer); this.radius = radius; }
    public String draw() { return renderer.render("circle(" + radius + ")"); }
}

final class Square extends Shape {
    private final int side;
    Square(Renderer renderer, int side) { super(renderer); this.side = side; }
    public String draw() { return renderer.render("square(" + side + ")"); }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.gof.structural.bridge;

/** Problem: support Vector and Raster rendering without exploding Shape×Renderer subclasses. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Bridge problem: shape x renderer ===");
    System.out.println("naive: VectorCircle, RasterCircle, VectorSquare, RasterSquare...");
    System.out.println(BridgeShapes.vectorCircle(3));
    System.out.println(BridgeShapes.rasterSquare(4));
  }
}
```

Walkthrough Output

```
=== Bridge problem: shape x renderer ===
naive: VectorCircle, RasterCircle, VectorSquare, RasterSquare...
vector:circle(3)
raster:square(4)
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Bridge.

Related Patterns

- Contrast Bridge with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Bridge → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.